

COMPLEJIDAD DE ALGORITMOS

Profesor: Ing. Weber Federico



¿Cómo sé si un algoritmo es mejor que otro?

«El que tarda menos tiempo»



No sirve: mide la máquina, no el algoritmo.

*El mismo programa, en dos PCs distintas,
da dos tiempos distintos.*

Necesitamos una medida que NO dependa del hardware.

Depende de **la PC**

Una máquina nueva le gana a una vieja... aunque corra un algoritmo peor.

Depende del **compilador**

Mismas líneas, distinto binario, distinto tiempo.

Depende de **la carga**

Si la máquina está ocupada, el mismo código tarda más.

¿Cómo sé si un algoritmo es mejor que otro?

~~«El que tarda menos tiempo»~~



No sirve: mide la máquina, no el algoritmo.

*El mismo programa, en dos PCs distintas,
da dos tiempos distintos.*

Necesitamos una medida que NO dependa del hardware.

Depende de **la PC**

Una máquina nueva le gana a una vieja... aunque corra un algoritmo peor.

Depende del **compilador**

Mismas líneas, distinto binario, distinto tiempo.

Depende de **la carga**

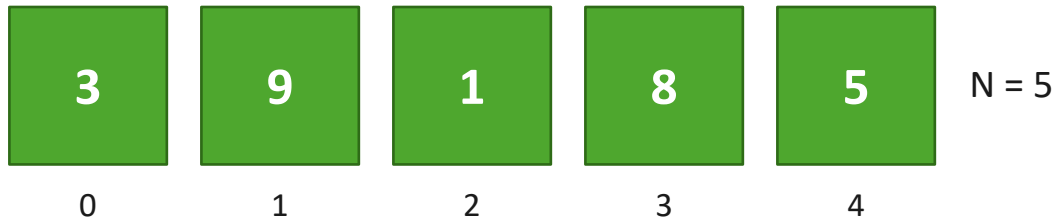
Si la máquina está ocupada, el mismo código tarda más.

Contar operaciones

Contamos las **operaciones** que hace el algoritmo en función del tamaño ***n*** de los datos.

Ejemplo sencillo — buscar el mayor de un vector: lo recorro una vez comparando contra el mayor que llevo.

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 } ;
```



Comparación:

¿este valor es mayor que el que guardé?
→ para N datos: $N - 1$ comparaciones.

Intercambio (swap):

mover dos valores; aparece al ordenar.

El tiempo cambia con la máquina. La cantidad de operaciones depende de N : por eso describe al algoritmo.

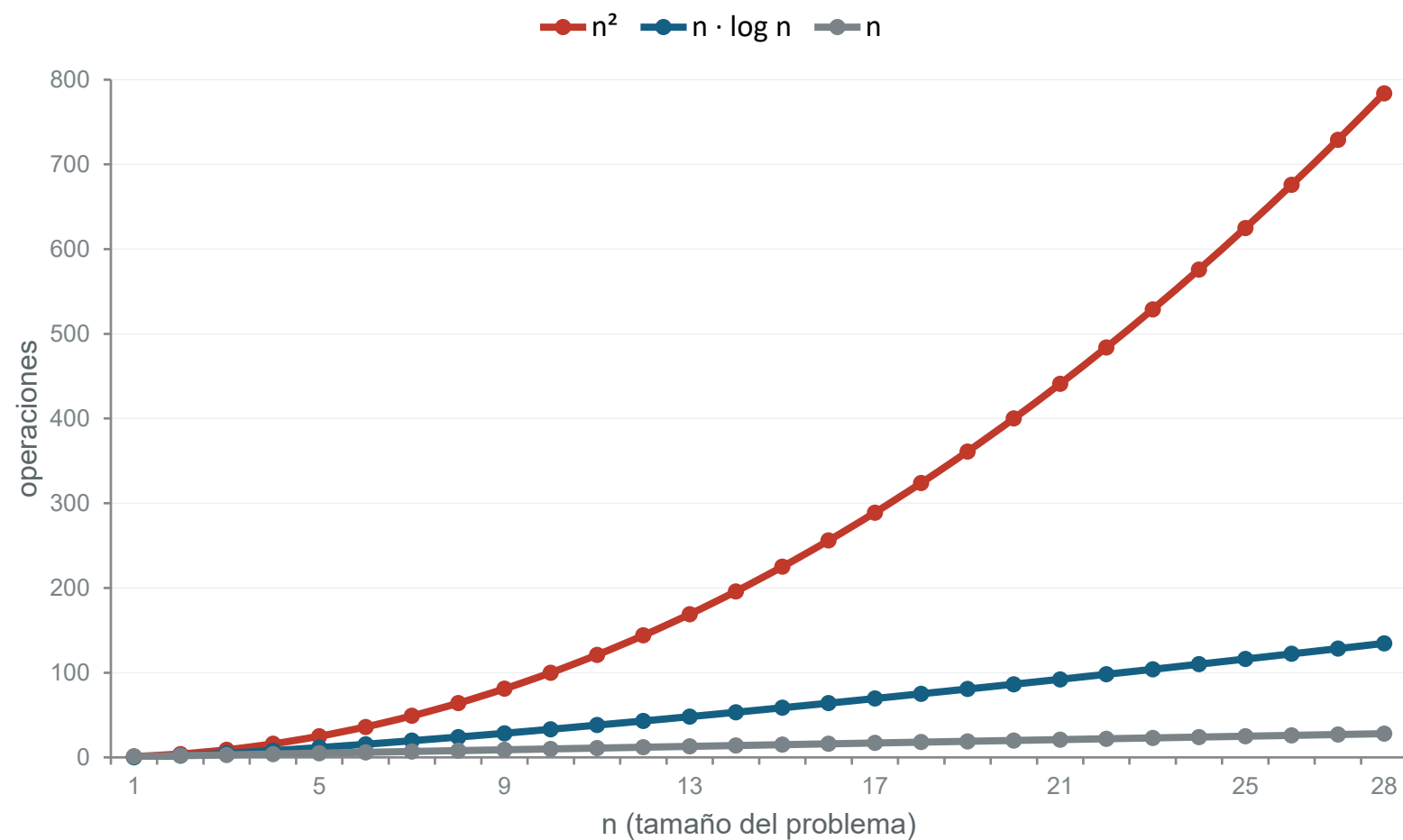
Lo que pasa cuando n crece

n (datos)	n	$n \cdot \log_2 n$	n^2
10	10	≈ 33	100
1.000	1.000	≈ 10.000	1.000.000
1.000.000	1.000.000	$\approx 20.000.000$	1.000.000.000.000

Con 1.000.000 de datos: n^2 = un billón de operaciones vs $n \cdot \log n \approx 20$ millones.

≈ 50.000 veces más operaciones. Por eso lo que importa es el ORDEN de crecimiento, no si hay un $\times 2$ o un $+5$ dando vueltas.

El orden importa (no la constante)



$$n < n \cdot \log n \ll n^2$$

Para n chico

las tres curvas casi se tocan.

Con pocos datos, casi cualquier método sirve.

La diferencia **ES NOTABLE** cuando n se hace grande.

Orden de complejidad de algoritmos

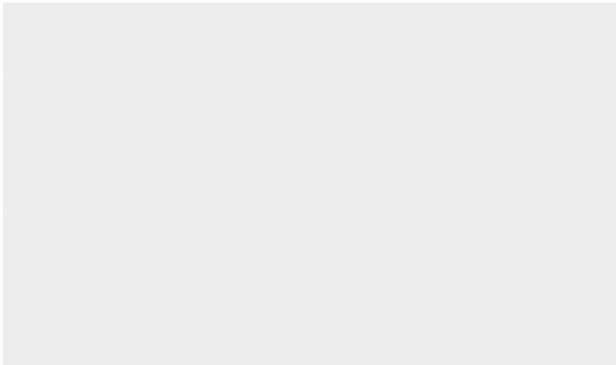
Categoría	Orden Complejidad	¿Qué significa?	Ejemplos
Constante	$O(1)$	Tiempo constante: no depende del tamaño	Acceder al primer elemento de una lista Acceder en forma directa a cualquier elemento de un vector
Sublineales	$O(\log n)$	Cada paso reduce el problema a la mitad	Búsqueda binaria en lista ordenada
	$O(\sqrt{n})$	Crece más rápido que $\log n$ pero menos que lineal; depende de la raíz del tamaño	Algoritmos en Árboles Búsqueda en estructuras jerárquicas
Lineales	$O(n)$	Crece proporcional al número de elementos	Recorrer una lista completa
	$O(n \log n)$	Combina recorrer y dividir el problema	Ordenar con MergeSort o QuickSort Ej: Ordenar cartas dividiéndolas en montones
Cuadráticas	$O(n^2)$	Por cada elemento revisas todos los demás	Ordenamiento burbujeo Ej: Torneo donde todos juegan contra todos
Cúbicas	$O(n^3)$	Por cada elemento revisas todos los demás en tres dimensiones	Algoritmos en matrices tridimensionales
Polinomial	$O(n^k), k \in \mathbb{N}$	Crece según una potencia fija; mayor que cúbico si $k > 3$	Algoritmos combinatorios simples
Exponenciales	$O(2^n)$	Cada elemento duplica el trabajo	Ej: Resolver problema de la mochila Ej: Probar todas las combinaciones de ropa
	$O(m^n)$	Crece aún más rápido: depende de dos variables (m y n)	Algoritmos en problemas multivariables; combinaciones con múltiples opciones
Factorial	$O(n!)$	Crece extremadamente rápido	Ej: Generar todas las permutaciones Ej: Probar todas las formas de sentar a 10 personas

Importancia del Orden de Complejidad

- La complejidad algorítmica es la brújula que guía la elección de algoritmos eficientes.
- Un algoritmo correcto no siempre es suficiente: debe ser eficiente cuando los datos crecen.
- La elección del orden de complejidad impacta en escalabilidad, recursos y experiencia de usuario.
- Con pocos datos, cualquier algoritmo puede servir; con millones, solo los eficientes sobreviven.
- La complejidad determina si un problema es resoluble en segundos... o en años.

Orden de complejidad de algoritmos

$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(n^k) < O(2^n) < O(m^n) < O(n!)$





Fin de la
presentación